Cyberscope

A TAC Security Company

Audit Report

EZManager

February 2026

Audited by © cyberscope

Table of Contents

Table of Contents 1
Risk Classification 3
Review 4
Audit Updates 4
Source Files 4
Overview 5
CLCore 5
Valuation 5
CLManager 6
UniswapAdapter 7
AerodromeAdapter 7
RebalancePlanner 8
Findings Breakdown 9
Diagnostics 10
DDA - Dust Decrement Accounting 12
Description 12
Recommendation 13
DRL - Dust Refund Loss 14
Description 14
Recommendation 15
TRR - Two-Hop Route Rejection 16
Description 16
Recommendation 17
ARS - Agnostic Route Size 18
Description 18
Recommendation 19
IPV - Inconsistent Position Valuation 20
Description 20
Recommendation 21
IDS - Ineffective Decimal Scaling 22
Description 22
Recommendation 23
PTAI - Potential Transfer Amount Inconsistency 24
Description 24
Recommendation 24
USA - Uniform Slippage Application 26
Description 26
Recommendation 27
MNOV - Missing NFT Ownership Validation 28

Description 28
Recommendation 29
CCR - Contract Centralization Risk 30
Description 30
Recommendation 32
CPI - Cross-Tick Price Impact 33
Description 33
Recommendation 34
EBA - Exceeding Budget Allocation 35
Description 35
Recommendation 36
FBP - Fee Bypass Path 37
Description 37
Recommendation 37
IDI - Immutable Declaration Improvement 38
Description 38
Recommendation 38
ISV - Inconsistent State Valuation 39
Description 39
Recommendation 40
MRV - Misleading Return Variables 41
Description 41
Recommendation 42
MEM - Missing Error Messages 43
Description 43
Recommendation 43
PFO - Pool Fee Omission 44
Description 44
Recommendation 44
PTWD - Possible TWAP Window Desync 45
Description 45
Recommendation 46
PSTP - Potential Stale TWAP Price 47
Description 47
Recommendation 47
PSM - Price Source Mismatch 49
Description 49
Recommendation 50
PFI - Principal Fee Inclusion 51
Description 51
Recommendation 53
STW - Short TWAP Window 54

Description 54
Recommendation 55
SCRS - Stale Cached Route Scores 56
Description 56
Recommendation 57
SDU - Stale Dust USDC 58
Description 58
Recommendation 59
UQA - Unconstrained Quoted Amount 60
Description 60
Recommendation 61
ZVE - Zero Value Estimation 62
Description 62
Recommendation 63
L04 - Conformance to Solidity Naming Conventions 64
Description 64
Recommendation 65
Functions Analysis 66
Inheritance Graph 84
Flow Graph 85
Summary 86
Disclaimer 87
About Cyberscope 88

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Commit	3ab47299dd4ad039c8a6a22827eb47f925e2224d

Audit Updates

Initial Audit	23 Dec 2025
Corrected Phase 2	4 Feb 2026

Source Files

Filename	SHA256
Valuation.sol	81c955a35f1aef5d445da4753ad73bb4628e63fa3bcc133ba731035caa142cd2
UniswapAdapter.sol	907f930dc9d19093fc947653fceabc5d56cf7471babd3d468b1221b1d7c02724
RebalancePlanner.sol	eac7e25563f7fac2f65eb0ab753ee487289c01dcec60c125614242a33f5e5745
CLManager.sol	a244278b879d839c94775fe1ac55ba9fbffc2cc1552072be8c4e7671ab870b02
CLCore.sol	27cf204d7512d5bb80313e94f15a19bed03170dad122f13789c2370da5c496ed
AerodromeAdapter.sol	96e37c7010b1a0a88666d289f0be54e342a58adf1c97200cd3da70ea05cf4372

Overview

The protocol implements a Concentrated Liquidity Management System for managing, accounting, and valuing liquidity positions across Uniswap V3 and Aerodrome Slipstream. The architecture separates state/accounting (CLCore) from pricing logic (Valuation), providing modular tracking of Liquidity Provider (LP) positions and their USDC-equivalent values.

CLCore

CLCore serves as the central ledger of the system. It is an upgradeable, owner-controlled contract responsible for tracking positions, managing balances, and calculating accrued fees.

Key Responsibilities:

- **Position Registry:** Maintains a mapping of Position structs that link a unique key to: LP NFT (tokenId), Owner address, Tick range (tickLower, tickUpper), Deposited principal (totalDepositedUSDC), Pending fees and dust balances.
- **Fee Accounting:** Calculates accrued fees for positions by comparing the pool's pending fees against the position's snapshots. This provides consistent fee tracking across Uniswap V3 and Aerodrome pools.
- **Dust Management:** Tracks residual USDC that is not currently deployed in a liquidity range, ensuring precise accounting of user balances.
- **Position Lifecycle Management:** Supports registration, deregistration, and updating of positions, including adjustments to total deposited USDC.

Valuation

Valuation is a view-only oracle contract that computes USDC-equivalent values for tokens held in the system.

Key Responsibilities:

- **USDC Valuation:** Converts token amounts into USDC-equivalent values.

- **Routing Logic:** Implements routing to identify the best pool based on liquidity depth scores. Supports direct USDC routing, and two-hop connector paths (via WETH or other bridge tokens).
- **Multi-DEX Support:** Handles both standard Uniswap V3 and Aerodrome pools, including different tick spacing and fee structures.
- **Liquidity Thresholds:** Ignores pools below a configurable minimum liquidity threshold.
- **TWAP Calculation:** Computes a short (default 60-second) Time-Weighted Average Price for price stability.
- **Route Caching:** Maintains cached edge scores for efficient route lookups, refreshed via `refreshAll()`.

CLManager

CLManager is the primary user-facing contract that orchestrates position operations. It coordinates between CLCore, Valuation, adapters, and the RebalancePlanner.

Key Responsibilities:

- **Position Opening:** Accepts USDC deposits, deducts protocol fees, seeds token pairs via adapters, plans rebalances, mints NFT positions, and registers them in CLCore.
- **Position Exiting:** Unwinds positions, converts tokens back to USDC, collects fees, and returns funds to users.
- **Fee Compounding:** Collects accrued trading fees and reinvests them into existing positions.
- **Collateral Management:** Supports adding and removing collateral from positions.
- **Range Changes:** Allows repositioning of liquidity to different tick ranges.
- **Bot Operations:** Supports automated operations via allowlisted bots with configurable bot fees.

Technical Highlights:

- **Second Pass Logic:** Attempts to deploy leftover USDC after initial minting if above configurable thresholds.
- **Slippage Budget Management:** Tracks remaining loss budget across multi-step operations.

- **Batch Operations:** Supports batch compounding and fee collection with configurable limits.

UniswapAdapter

UniswapAdapter is the DEX adapter implementation for Uniswap V3 and PancakeSwap V3 pools. It implements the `ICLDexAdapter` interface.

Key Responsibilities:

- **Position Minting:** Creates new concentrated liquidity positions via the NonfungiblePositionManager.
- **Liquidity Management:** Adds and removes liquidity from existing positions.
- **Token Swaps:** Executes swaps via the SwapRouter with TWAP-based slippage protection.
- **Fee Collection:** Collects accrued trading fees from positions.
- **Seed Pair Conversion:** Converts USDC into position tokens, supporting direct routes and bridge token swaps.

Technical Highlights:

- **TWAP-Based Slippage:** Uses pool TWAP for calculating expected outputs and minimum amounts.
- **Remaining Loss Budget Pattern:** Tracks cumulative slippage across multi-step operations.
- **PancakeSwap Support:** Includes `isPancakeSwap` flag to handle router interface differences.
- **Route Discovery:** Delegates to Valuation for best route selection based on cached depth scores.

AerodromeAdapter

AerodromeAdapter is the DEX adapter implementation for Aerodrome Slipstream pools on Base. It implements the `ICLDexAdapter` interface with Aerodrome-specific handling.

Key Responsibilities:

- **Position Minting:** Creates new concentrated liquidity positions via the Slipstream NonfungiblePositionManager.
- **Liquidity Management:** Adds and removes liquidity from existing positions.
- **Token Swaps:** Executes swaps via the Slipstream SwapRouter with TWAP-based slippage protection.
- **Fee Collection:** Collects accrued trading fees from positions.
- **Seed Pair Conversion:** Converts USDC into position tokens for Aerodrome pools.

Technical Highlights:

- **Tick Spacing vs Fee:** Aerodrome uses tick spacing instead of fee tiers for pool identification.
- **Slipstream Interfaces:** Uses Aerodrome-specific interfaces (ISlipstreamSwapRouter, ISlipstreamNonfungiblePositionManager).
- **TWAP Integration:** Computes TWAP via pool observations for price stability.

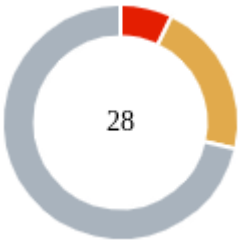
RebalancePlanner

RebalancePlanner computes optimal swap amounts to achieve the desired token ratio for a given tick range using iterative numerical methods.

Key Responsibilities:

- **Swap Amount Calculation:** Determines optimal swap direction and amount to balance token holdings for a target liquidity range.
- **Iterative Solver:** Uses a damped Newton's method (Iterative Value Decomposition) to converge on optimal amounts.
- **Price Impact Modeling:** Accounts for price movement during swaps when computing target ratios.

Findings Breakdown



Critical	2
Medium	6
Minor / Informative	20

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	2	0	0	0
Medium	6	0	0	0
Minor / Informative	20	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
Critical	DDA	Dust Decrement Accounting	Unresolved
Critical	DRL	Dust Refund Loss	Unresolved
Medium	TRR	Two-Hop Route Rejection	Unresolved
Medium	ARS	Agnostic Route Size	Unresolved
Medium	IPV	Inconsistent Position Valuation	Unresolved
Medium	IDS	Ineffective Decimal Scaling	Unresolved
Medium	PTAI	Potential Transfer Amount Inconsistency	Unresolved
Medium	USA	Uniform Slippage Application	Unresolved
Minor / Informative	MNOV	Missing NFT Ownership Validation	Unresolved
Minor / Informative	CCR	Contract Centralization Risk	Unresolved
Minor / Informative	CPI	Cross-Tick Price Impact	Unresolved
Minor / Informative	EBA	Exceeding Budget Allocation	Unresolved
Minor / Informative	FBP	Fee Bypass Path	Unresolved

●	IDI	Immutable Declaration Improvement	Unresolved
●	ISV	Inconsistent State Valuation	Unresolved
●	MRV	Misleading Return Variables	Unresolved
●	MEM	Missing Error Messages	Unresolved
●	PFO	Pool Fee Omission	Unresolved
●	PTWD	Possible TWAP Window Desync	Unresolved
●	PSTP	Potential Stale TWAP Price	Unresolved
●	PSM	Price Source Mismatch	Unresolved
●	PFI	Principal Fee Inclusion	Unresolved
●	STW	Short TWAP Window	Unresolved
●	SCRS	Stale Cached Route Scores	Unresolved
●	SDU	Stale Dust USDC	Unresolved
●	UQA	Unconstrained Quoted Amount	Unresolved
●	ZVE	Zero Value Estimation	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved

DDA - Dust Decrement Accounting

Criticality	Critical
Location	CLManager.sol#L1239,801,1206
Status	Unresolved

Description

The `withdrawDust` function reduces the `totalDepositedUSDC` value by the amount of dust withdrawn using `CORE.adjustTotalDeposited(key, -int256(dust))`. This behavior is consistent when the dust originates from an initial deposit, such as through the `openPosition` function, where the full deposit amount is included in `totalDepositedUSDC`.

However, in other scenarios, specifically within the `compound` and `changeRange` functions, dust is added via `_addDust` without a corresponding increase to `totalDepositedUSDC`. As a result, reducing `totalDepositedUSDC` via `withdrawDust` within these contexts leads to an inaccurate accounting of deposited funds.

```shell
function withdrawDust(bytes32 key) external nonReentrant
onlyKeyOwner(key) whenNotPaused {
    Position memory p = CORE.getPosition(key);
    uint256 dust = CORE.withdrawDustForPosition(key,
address(this), type(uint256).max);
    if (dust == 0) revert ZeroAmount();
    CORE.adjustTotalDeposited(key, -int256(dust));
    USDC.safeTransfer(p.owner, dust);

}
```

Recommendation

The team is advised to review the accounting logic to ensure that adjustments to `totalDepositedUSDC` within the `withdrawDust` function accurately reflect the source and amount of dust being withdrawn. This verification helps maintain consistency in the contract's internal accounting and prevents potential discrepancies.

DRL - Dust Refund Loss

Criticality	Critical
Location	UniswapAdapter.sol#L1292 AerodromeAdapter.sol#L1268
Status	Unresolved

Description

The `_returnDust` function employs a delta-based refund mechanism that calculates the amount of USDC to return by measuring the change in the contract's USDC balance during execution. However, this approach fails when USDC is one of the position tokens, such as in a WETH/USDC pool.

In such cases, any pre-existing USDC held by the contract before the function call is included in the `usdcBefore` value. Since the function checks whether each token is different from USDC before attempting a conversion, it skips converting USDC itself. As a result, any leftover USDC from the minting process is not converted or refunded, and the delta calculation (`usdcAfter - usdcBefore`) does not account for this pre-existing balance. This leads to an under-refund of USDC to the recipient.

```shell
function _returnDust(
    address token0,
    address token1,
    address usdc,
    address recipient,
    uint256 remainingLossUSDC
) internal returns (uint256 leftoverUSDC, uint256
remainingLossUSDCOut) {
    ...}
```

Recommendation

Adapters should track and return any unconsumed USDC separately when USDC is used as one of the position tokens. This ensures accurate accounting and prevents potential misallocation or loss of funds during operation settlement.

TRR - Two-Hop Route Rejection

Criticality	Medium
Location	UniswapAdapter.sol#L776 AerodromeAdapter.sol#L760
Status	Unresolved

Description

The `_swapUSDCToToken` function requires the route returned by `_getBestPath` to be a direct path. Specifically, it checks that the route exists, is marked as direct, and that `poolA` is not the zero address:

```shell
TokenRoute memory r = _getBestPath(usdc, tokenOut);

if (!r.exists || !r.isDirect || r.poolA == address(0))
revert PoolNotFound();
```

The `_getBestPath` function delegates to `Valuation._bestRouteCached`, which evaluates both direct and 2-hop routes (USDC → connector → tokenOut) and selects the one with the higher liquidity score. If a 2-hop route is superior, the returned route will have `poolB != address(0)` and `isDirect = false`.

As a result, when a 2-hop route offers better liquidity, the adapter still reverts with `PoolNotFound`, even though a valid and more optimal route exists. This behavior prevents `openPosition` from proceeding in cases where the 2-hop connector path is more liquid than the direct USDC-to-bridge pool.

Recommendation

Support for 2-hop swaps should be added to the `_swapUSDCToToken` function when `isDirect` is set to `false`. This enhancement would allow the function to perform sequential swaps, first converting USDC to an intermediate connector token, and then swapping the connector token to the final `tokenOut`.

ARS - Agnostic Route Size

Criticality	Medium
Location	Valuation.sol#L227
Status	Unresolved

Description

The `_bestRouteCached` function selects the route with the highest depth score (`scoreUSDC`), which reflects a pool's ability to absorb sell pressure without significant price impact. However, the actual token amount being evaluated is never compared against this depth capacity.

As a result, when a large token amount is routed through a pool with insufficient depth, the system still prices the entire amount using a flat time-weighted average price (TWAP). This approach ignores the potential for substantial slippage, leading to inaccurate valuations. Although depth data is available, it is used solely for ranking routes and not for assessing whether a route can realistically handle the trade size.

```shell
function usdcValue(address dex, address token, uint256
amount) external view returns (uint256) {
    if (amount == 0) return 0;
    if (token == USDC) return amount;
    if (CORE == address(0)) revert CoreNotSet();
    if (!ICLCore(CORE).allowedDexes(dex)) revert
DexNotAllowed();
    DexContext memory ctx = _ctx(dex);
    if (ctx.factory == address(0)) revert
DexFactoryNotFound();

    (address poolA, address poolB, ) =
_bestRouteCached(dex, token, USDC);
    if (poolA == address(0)) revert RouteNotFound();
```

```
uint256 outScaled = _quoteRoute(poolA, poolB, token, USDC,
amountScaled);}
```

Recommendation

The team could consider validating that the amount being evaluated does not significantly exceed the route's depth score, or applying a discount factor to amounts that surpass the pool's capacity. This approach can help ensure more accurate pricing and prevent unrealistic valuations in scenarios where liquidity is insufficient.

IPV - Inconsistent Position Valuation

Criticality	Medium
Location	CLCore.sol#L829,1075
Status	Unresolved

Description

The `_positionValueUSDCSingle` function calculates the value of a position using two different pricing mechanisms, which may lead to inconsistencies during periods of high market volatility.

```shell
function _positionValueUSDCSingle(Position memory p)
internal view returns (uint256) {
    if (p.owner == address(0) || p.tokenId == 0) revert
PositionNotFound();

    (uint256 amt0, uint256 amt1, , ) =
_spotAmountsForPosition(p);

    uint256 v0 = _valueTokenToUSDC(p.dex, p.token0, amt0);
    uint256 v1 = _valueTokenToUSDC(p.dex, p.token1, amt1);
    return v0 + v1 + p.dustUSDC;

}
```

Within `_spotAmountsForPosition`, the function derives token amounts using the spot price obtained from `_readSlot0`:

```shell
(uint160 sqrtPriceX96,) = _readSlot0(pool);

(amt0, amt1) =
LiquidityAmounts.getAmountsForLiquidity(
    sqrtPriceX96,
    sqrtRatioAX96,
    sqrtRatioBX96,
    liquidity

);
```

However, when converting these token amounts to USDC, the `_valueTokenToUSDC` function relies on the `VALUATION.usdcValue` method, which uses a time-weighted average price (TWAP). This mismatch can result in inconsistent or misleading position values.

Recommendation

The team is advised to use a consistent price source throughout the entire position valuation calculation. This ensures uniformity in pricing data, reduces the risk of discrepancies, and improves the accuracy and reliability of the valuation process.

IDS - Ineffective Decimal Scaling

Criticality	Medium
Location	Valuation.sol#L230
Status	Unresolved

Description

The decimal scaling logic in the `usdcValue` function is redundant and has no effect on the final result. For tokens with more than 18 decimals, the function attempts to scale the input amount up by a factor (`scale`), perform a price quote, and then scale the output down by the same factor. However, because the `_quoteThroughPool` function uses a linear time-weighted average price (TWAP) model, where the output is calculated as `amountIn × price` , the scaling operations cancel each other out.

This can be illustrated as follows:

- `amountScaled = amount × scale`
- `outScaled = amountScaled × price = amount × scale × price`
- `result = outScaled / scale = amount × price`

As a result, the final output is simply `amount × price` , regardless of the scaling. This renders the scaling logic ineffective.

```shell
uint8 tokenDecimals = IERC20Metadata(token).decimals();
uint256 scale = 1;
if (tokenDecimals > 18) {
    uint256 k = uint256(tokenDecimals - 18);
    if (k > 18) k = 18;
    scale = 1;
    for (uint256 i = 0; i < k; ++i) scale *= 10;
}
```

```
uint256 amountScaled = scale == 1 ? amount :
FullMath.mulDiv(amount, scale, 1);
uint256 outScaled = _quoteRoute(poolA, poolB, token, USDC,
amountScaled);

if (scale == 1) return outScaled;

return FullMath.mulDiv(outScaled, 1, scale);
```

Recommendation

Consider revising the unused scaling logic to reduce gas consumption and simplify the contract's codebase. Eliminating dead code minimizes execution overhead and improves maintainability.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Medium
Location	UniswapAdapter.sol#L324,325
Status	Unresolved

Description

The `transfer()` and `transferFrom()` functions are used to transfer a specified amount of tokens to an address. The fee or tax is an amount that is charged to the sender of an ERC20 token when tokens are transferred to another address. According to the specification, the transferred amount could potentially be less than the expected amount. This may produce inconsistency between the expected and the actual behavior.

```shell
_requireBalance(token0, amount0);

_requireBalance(token1, amount1);
```

Recommendation

The team is advised to take into consideration the actual amount that has been transferred instead of the expected.

It is important to note that an ERC20 transfer tax is not a standard feature of the ERC20 specification, and it is not universally implemented by all ERC20 contracts. Therefore, the contract could produce the actual amount by calculating the difference between the transfer call.

```
Actual Transferred Amount = Balance After Transfer - Balance Before Transfer
```

USA - Uniform Slippage Application

Criticality	Medium
Location	UniswapAdapter.sol#L1188 AerodromeAdapter.sol#L1164
Status	Unresolved

Description

The `_computeSlippageMins` function calculates a slippage-adjusted minimum amount for each token by first determining the total expected value in USDC and then applying a uniform loss percentage (`lossBps`) to both tokens. This percentage is derived from the combined USDC value of `token0` and `token1` :

```shell
uint256 expectedValueUSDC = _usdcValue(usdc, token0,
used0Pred) + _usdcValue(usdc, token1, used1Pred);
uint256 lossBps = _lossBpsFromBudget(remainingLossUSDC,
expectedValueUSDC);

min0 = used0Pred == 0 ? 0 : (used0Pred * (BPS - lossBps)) /
BPS;
if (min0 > balance0) min0 = balance0;

min1 = used1Pred == 0 ? 0 : (used1Pred * (BPS - lossBps)) /
BPS;

if (min1 > balance1) min1 = balance1;
```

However, when the two tokens contribute unequally to the total value, for example, if `token0` is worth $1 and `token1` is worth $10, applying the same percentage reduction to both can misrepresent the intended slippage distribution for each token. This may result

in one of the `min` values exceeding what the pool can accept at the current tick position, potentially causing the `NPM.mint()` function to revert.

Recommendation

The team is advised to adjust the slippage calculation to account for each token's individual contribution to the total USDC value. This ensures that the slippage tolerance for each token is weighted according to its proportion of the overall position value, leading to more accurate and balanced execution.

MNOV - Missing NFT Ownership Validation

Criticality	Minor / Informative
Location	CLCore.sol#L362
Status	Unresolved

Description

The `registerPosition` function stores the provided tokenId without verifying that the CLCore contract actually owns the corresponding NFT. While the CLManager contract is responsible for transferring the minted token to CLCore before invoking registerPosition, the CLCore contract itself does not perform an explicit ownership check. This reliance on external behavior without internal validation may lead to inconsistencies or vulnerabilities if the expected transfer does not occur as intended.

```shell
function registerPosition(RegisterParams calldata
params)
    external
    onlyManager
    whenNotPaused
    nonReentrant
    returns (bytes32 key)
{
   ...

}
```

Recommendation

Consider adding NFT ownership verification within the `CLCore` contract to ensure that it holds the token before proceeding with registration. This check helps prevent unauthorized or invalid registrations by confirming that the contract has rightful control over the NFT at the time of the operation.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	CLCore.sol#L161,170,176,183,192,217,254,260,267,303,328,334,341,348,352 CLManager.sol#L211,217,223,229,235,241,247,251 UniswapAdapter.sol#L133,137,141,147,153,161,167 AerodromeAdapter.sol#L124,128,132,138,144,152,158 Valuation.sol#L73,81,87,97 RebalancePlanner.sol#L111,118,123,129
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```shell
function setValuation(address valuation) external onlyOwner {...}
function setProtocolReserve(address reserve) external onlyOwner {...}
function setManager(address mgr) external onlyOwner {...}
function setGuardian(address g) external onlyOwner {...}
function addAllowedDex(address dex) external onlyOwner {...}
function removeAllowedDex(address dex) external onlyOwner {...}
function addBot(address bot) external onlyOwner {...}
function removeBot(address bot) external onlyOwner {...}
function setPoolStatus(address pool, PoolStatus status) public onlyOwner {...}
function setBridgeTokens(address[] calldata bridges) external onlyOwner {...}
```

```
function setZeroFeeWallet(address wallet, bool isZeroFee)
external onlyOwner {...}
function setProtocolFeeBps(uint16 newBps) external
onlyOwner {...}
function setBotFeeBps(uint16 newBps) external onlyOwner
{...}
function pause() external onlyGuardian {...}

function unpause() external onlyGuardian {...}
```

```Shell
function setPlanner(address planner_) external onlyOwner
{...}
function setValuation(address valuation_) external
onlyOwner {...}
function setGuardian(address g) external onlyOwner {...}
function setMinimumOpenUSDC(uint256 min_) external
onlyOwner {...}
function setMaxBatchKeys(uint256 maxKeys_) external
onlyOwner {...}
function setSecondPassParams(uint256 thresholdBps_, uint256
minUsdc_) external onlyOwner {...}
function pause() external onlyGuardian {...}

function unpause() external onlyGuardian {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

CPI - Cross-Tick Price Impact

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L1019 AerodromeAdapter.sol#L993
Status	Unresolved

Description

The `_minOutFromPool` function estimates the expected output of a swap using the time-weighted average price (TWAP), assuming a constant price throughout the transaction:

```shell
uint256 amountInPostFee = FullMath.mulDiv(amountIn,
(1_000_000 - fee), 1_000_000);

uint160 sqrtPriceTwapX96 =
TickMath.getSqrtRatioAtTick(meanTick);
uint256 priceX96 = FullMath.mulDiv(
    uint256(sqrtPriceTwapX96),
    uint256(sqrtPriceTwapX96),
    FixedPoint96.Q96
);

uint256 expectedOut = zeroForOne
    ? FullMath.mulDiv(amountInPostFee, priceX96,
FixedPoint96.Q96)

    : FullMath.mulDiv(amountInPostFee, FixedPoint96.Q96,

priceX96);
```

However, in Uniswap V3's concentrated liquidity model, swaps often traverse multiple price ticks. Each tick crossing shifts the price unfavorably for the swapper, resulting in less output than a simple TWAP-based calculation would predict.

As a result, the `minOut` value may be set too high, causing valid swaps to revert.

Recommendation

Consider using the Quoter contract to simulate the actual swap output, as it accounts for the effects of crossing multiple price ticks.

EBA - Exceeding Budget Allocation

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L860 AerodromeAdapter.sol#L850
Status	Unresolved

Description

The system processes multiple swap operations by sequentially passing the remaining loss budget, denominated in USDC, through each step. Each swap is permitted to consume up to the full amount of the remaining budget:

```shell
uint256 expectedOut = _expectedOutFromPool(pool, tokenIn, amountIn);
uint256 expectedOutUSDC_ = _expectedOutUSDC(usdc, tokenOut, expectedOut);

uint256 minOut = _minOutFromLossBudget(expectedOut, expectedOutUSDC_, remainingLossUSDC);
```

After each swap, the actual loss consumed is calculated and subtracted from the remaining budget:

```shell
uint256 usedLossUSDC = _usedLossUSDC(expectedOut, out, expectedOutUSDC_);
if (usedLossUSDC > remainingLossUSDC) usedLossUSDC =
```

```
remainingLossUSDC;

remainingLossUSDCOut = remainingLossUSDC -
usedLossUSDC;
```

If an early swap executes at the minimum acceptable output (`minOut`), it may consume the entire loss budget. As a result, subsequent swaps are left with little to no tolerance for slippage, effectively requiring the output to match the expected value exactly. Any deviation in price then causes these later swaps to revert.

Recommendation

The team is advised to pre-allocate portions of the slippage budget across expected operations, rather than assigning 100% of the budget to each step sequentially. This approach ensures more accurate control over cumulative slippage and reduces the risk of exceeding the overall budget during multi-step processes.

FBP - Fee Bypass Path

Criticality	Minor / Informative
Location	CLManager.sol#L573
Status	Unresolved

Description

The `returnNft` function does not apply any protocol or bot fees. Instead, it invokes `CORE.returnPosition`, which transfers both the NFT and any residual token balances ("dust") directly back to the position owner. In contrast, the `exitPosition` function deducts a bot fee (`botFeeBps`) from the proceeds. Because `returnNft` can be called by the position owner, users have the option to bypass all associated fees by choosing this method.

```shell
function returnNft(bytes32[] calldata keys)
    external
    nonReentrant
{
    ...

}
```

Recommendation

The team can consider enforcing fee collection on this exit path to maintain consistency with other withdrawal mechanisms. This ensures uniform behavior across all withdrawal processes and helps prevent potential discrepancies in fee handling.

IDI - Immutable Declaration Improvement

Criticality	Minor / Informative
Location	Valuation.sol#L70 UniswapAdapter.sol#L127,128 AerodromeAdapter.sol#L119
Status	Unresolved

Description

The contract declares state variables that their value is initialized once in the constructor and are not modified afterwards. The `immutable` is a special declaration for this kind of state variables that saves gas when it is defined.

```shell
USDC
uniswapQuoter
isPancakeSwap

aerodromeQuoter
```

Recommendation

By declaring a variable as immutable, the Solidity compiler is able to make certain optimizations. This can reduce the amount of storage and computation required by the contract, and make it more gas-efficient.

ISV - Inconsistent State Valuation

Criticality	Minor / Informative
Location	CLManager.sol#L801,1151
Status	Unresolved

Description

The `totalDepositedUSDC` field can become outdated following certain operations that alter a position's value without updating this tracking variable.

In the `changeRange` function, when a position is restructured, potentially incurring slippage losses, only the NFT metadata is updated:

```shell
CORE.updateTokenMetadata(key, newTokenId,
newTickLower, newTickUpper);
```

Similarly, in the `compound` function, accrued fees are reinvested into the position, but `totalDepositedUSDC` remains unchanged:

```shell
_addDust(key, leftoverUSDC);
```

Neither of these operations invokes `CORE.adjustTotalDeposited`, which leads to a growing discrepancy between `totalDepositedUSDC` and the actual value of the position. The stale `totalDepositedUSDC` may still be referenced potentially leading to misleading or incorrect information.

Recommendation

The team is advised to update the `totalDepositedUSDC` variable to reflect value changes following position restructuring. This ensures the variable accurately represents the current state of deposits.

MRV - Misleading Return Variables

Criticality	Minor / Informative
Location	CLManager.sol#L702,703
Status	Unresolved

Description

The `compoundFees` function processes multiple positions in a batch and returns three values: `compoundedUSDCTotal`, `used0Total`, and `used1Total`. While `compoundedUSDCTotal` is consistently denominated in USDC and provides meaningful data, the `used0Total` and `used1Total` values are dependent to the processed position.

These two totals are calculated by summing the `used0` and `used1` values from each position, without accounting for the fact that each position may involve entirely different token pairs. As a result, `used0Total` and `used1Total` would retrieve misleading data when positions with different token pairs are batched together.

```shell
used0Total += used0;

used1Total += used1;
```

Recommendation

If batched operations are to be processed, it is advisable to remove `used0Total` and `used1Total` from the return values, retaining only `compoundedUSDCTotal`, which is consistently denominated in USDC. Alternatively, the team is advised to enforce constant token pair structures across all keys in the batch to ensure consistency and prevent misinterpretation.

MEM - Missing Error Messages

Criticality	Minor / Informative
Location	RebalancePlanner.sol#L29 CLManager.sol#L23
Status	Unresolved

Description

The contract is missing error messages. Specifically, there are no error messages to accurately reflect the problem, making it difficult to identify and fix the issue. As a result, the users will not be able to find the root cause of the error.

```shell
require(usdcDecimals >= 3)
```

Recommendation

The team is suggested to provide a descriptive message to the errors. This message can be used to provide additional context about the error that occurred or to explain why the contract execution was halted. This can be useful for debugging and for providing more information to users that interact with the contract.

PFO - Pool Fee Omission

Criticality	Minor / Informative
Location	Valuation.sol#L664
Status	Unresolved

Description

The `_quoteThroughPool` function calculates token output amounts using the time-weighted average price (TWAP) without accounting for Uniswap pool fees, which typically range from 0.05% to 1%. As a result, the returned quote reflects a pre-fee value that is higher than what would be received in an actual swap. This leads to a consistent overestimation of token values, and the discrepancy becomes more significant in multi-hop swaps, where fees are applied at each step.

```shell
function _quoteThroughPool(address pool, address tokenIn,
address tokenOut, uint256 amountIn)
    internal
    view
    returns (uint256)
{...}
```

Recommendation

Consider retrieving the pool's fee tier and subtracting it from the quoted amount. This adjustment ensures that the valuation reflects actual transaction costs, resulting in more accurate and realistic pricing.

PTWD - Possible TWAP Window Desync

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L34 UniswapAdapter.sol#L161 AerodromeAdapter.sol#L32 Valuation.sol#L39,81
Status	Unresolved

Description

Both the adapter and the Valuation contract define their own configurable TWAP (Time-Weighted Average Price) window through separate TWAP_SECONDS variables and independent setter functions:

```shell
// UniswapAdapter.sol
uint32 public TWAP_SECONDS = 60;

function setTwapSeconds(uint32 s) external onlyOwner
{
    if (s < 30 || s > 3600) revert InvalidParam();
    TWAP_SECONDS = s;
}

uint32 public TWAP_SECONDS = 60;

function setTWAPSeconds(uint32 seconds_) external
onlyOwner {
    if (seconds_ < 30 || seconds_ > 3600) revert
InvalidTWAPSeconds();
    TWAP_SECONDS = seconds_;
}
```

Because these values are managed independently, updating the TWAP window in one contract without synchronizing it with the other can lead to discrepancies. This results in the adapter and Valuation contract computing prices over different time intervals for the same pool, which may cause inconsistent slippage calculations and inaccurate token valuations.

Recommendation

The team must ensure that the `TWAP_SECONDS` parameter is updated consistently across all relevant contracts. This synchronization is essential to maintain consistent time-weighted average price calculations across the system.

PSTP - Potential Stale TWAP Price

Criticality	Minor / Informative
Location	Valuation.sol#L706
Status	Unresolved

Description

The `_twapSqrtPriceX96` function ensures that the `observe()` call does not fail but does not verify whether the underlying pool has experienced recent trading activity:

```shell
try IUniswapV3Pool(pool).observe(secondsAgosPrice)
    returns (int56[] memory tCumulatives,
uint160[] memory splPrice)
{
    tickCumulatives = tCumulatives;
    splCumulativesPrice = splPrice;
} catch {
    revert TWAPObservationFailed();

}
```

If no swaps have occurred for a prolonged period, the tick cumulative remains constant at the last recorded tick. As a result, the Time-Weighted Average Price (TWAP) calculation may return a seemingly valid price that is actually based on outdated market data.

Recommendation

Consider adding a staleness check based on the pool's last swap timestamp or by verifying that the tick cumulative has changed within a predefined timeframe. This helps ensure that

the data used for calculations is recent and reliable, reducing the risk of relying on outdated or manipulated information.

PSM - Price Source Mismatch

Criticality	Minor / Informative
Location	RebalancePlanner.sol#L476 UniswapAdapter.sol#L1076 AerodromeAdapter.sol#L1051
Status	Unresolved

Description

The `RebalancePlanner` contract calculates swap amounts using the instantaneous spot price retrieved from the `slot0()` function:

```shell
function _safeGetSqrtPrice(address pool) internal view
returns (uint160 sqrtPriceX96) {
    (bool success, bytes memory data) =
pool.staticcall(abi.encodeWithSignature("slot0()"));
    if (!success || data.length < 32) revert Slot0Failed();

    sqrtPriceX96 = abi.decode(data, (uint160));

}
```

In contrast, the `UniswapAdapter` and `AerodromeAdapter` contracts use a time-weighted average price (TWAP) for slippage calculations:

```shell
uint32[] memory secondsAgos = new uint32[](2);
secondsAgos[0] = TWAP_SECONDS;
```

```
secondsAgos[1] = 0;

(int56[] memory tickCumulatives, ) =
p.observe(secondsAgos);
```

This inconsistency between using the spot price for planning swaps and the TWAP for slippage protection can lead to issues during periods of high market volatility. When the spot price significantly deviates from the TWAP, the calculated swap amounts may not achieve the intended ratio. As a result, slippage protections may either fail to trigger or permit more slippage than expected.

Recommendation

The team is advised to use a consistent price source across both the `RebalancePlanner` and `UniswapAdapter` components. This ensures accurate swap planning and reliable slippage protection, reducing the risk of discrepancies that could lead to suboptimal trades or unintended losses.

PFI - Principal Fee Inclusion

Criticality	Minor / Informative
Location	CLManager.sol#L294,811
Status	Unresolved

Description

Both the `openPosition` and `addCollateral` functions calculate a protocol fee from the user's deposit and transfer it to the reserve. However, they record the total deposited USDC using the gross deposit amount, which includes the protocol fee, rather than the net amount that is actually added to the position's liquidity.

In `openPosition`, the protocol fee is deducted from the user's deposit, and only the net amount is used for liquidity provision:

```shell
uint256 protocolFeeUSDC =
_protocolFeeAmount(usdcAmount, protocolFeeBps_);
uint256 seedNet = usdcAmount - protocolFeeUSDC;

if (protocolFeeUSDC > 0) {
    USDC.safeTransfer(reserve, protocolFeeUSDC);

}
```

Despite this, the full deposit amount—including the protocol fee—is recorded when registering the position:

```shell
RegisterParams memory params = RegisterParams({
    // ...
    totalDepositedUSDC: usdcAmount,
    // ...

});
```

A similar pattern occurs in `addCollateral`, where the protocol fee is deducted and only the net amount is added to the position, but the gross amount is still recorded:

```shell
uint256 add = usdcAmount;
uint256 protocolFeeUSDC = _protocolFeeAmount(add,
protocolFeeBps_);
uint256 net = add - protocolFeeUSDC;
// ...protocol fee sent to reserve, only net goes
to liquidity...

uint256 depositedUSDC = add;  // includes protocol
fee

CORE.adjustTotalDeposited(key,
int256(depositedUSDC));
```

This results in a discrepancy between the recorded principal and the actual capital deployed in the position. Any downstream logic that relies on `totalDepositedUSDC` to calculate profit or loss will misrepresent returns, as it includes funds that were never allocated to the position.

Recommendation

It is advised to track the principal actually deployed in the position. The team could clarify the intended semantics of totalDepositedUSDC and ensure consistent usage across the codebase.

STW - Short TWAP Window

Criticality	Minor / Informative
Location	Valuation.sol#L39,81
Status	Unresolved

Description

The contract allows the owner to configure the time-weighted average price (TWAP) interval via the `TWAP_SECONDS` variable, which defaults to 60 seconds and can be set to any value between 30 and 3600 seconds. On Ethereum mainnet, where blocks are produced approximately every 12 seconds, a 60-second TWAP relies on only about five oracle observations. This limited data set makes the price feed vulnerable to manipulation, as a well-capitalized attacker could influence prices over such a short window at a relatively low cost.

In decentralized finance (DeFi), it is common practice to use TWAP intervals of 30 minutes or longer to ensure sufficient resistance against price manipulation.

```shell
uint32 public TWAP_SECONDS = 60;

function setTWAPSeconds(uint32 seconds_) external
onlyOwner {
    if (seconds_ < 30 || seconds_ > 3600) revert
InvalidTWAPSeconds();
    TWAP_SECONDS = seconds_;
    emit TWAPSecondsUpdated(seconds_);

}
```

Recommendation

Consider increasing the minimum Time-Weighted Average Price (TWAP) window. A longer TWAP window reduces the impact of short-term price fluctuations, making it more difficult for attackers to manipulate the average price. This enhances the system's resistance to price manipulation and improves overall price reliability.

SCRS - Stale Cached Route Scores

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L278 AerodromeAdapter.sol#L264 Valuation.sol#L267
Status	Unresolved

Description

When both position tokens are bridge tokens, seedPairFromUSDC selects which bridge to use as the seed token by comparing cached liquidity scores:

```shell
else if (token0IsBridge && token1IsBridge) {
    uint256 score0 =
_bestAllowlistedLiqScore(usdc, token0);
    uint256 score1 =
_bestAllowlistedLiqScore(usdc, token1);
    if (score0 == 0 && score1 == 0) revert
UnsupportedPair();
    seedToken = score0 >= score1 ? token0 :
token1;

}
```

`_bestAllowlistedLiqScore` delegates to Valuation.getBestRoute, which reads from `edgeToConnector`, a cache populated by `refreshAll()` at discrete intervals. If liquidity conditions changed since the last refresh, the comparison may select a bridge with currently inferior liquidity, resulting in worse swap execution.

Recommendation

The team is advised to query the current on-chain state instead of relying on cached scores for seed token selection. This will ensure consistency of operations.

SDU - Stale Dust USDC

Criticality	Minor / Informative
Location	CLManager.sol#L811
Status	Unresolved

Description

The addCollateral function invokes _maybeSecondPass, which may return a small amount of leftover USDC that could not be added as liquidity. This leftover is discarded as returned value and not added as dust to the position. As a result, the leftover USDC remains in the Manager contract without any tracking or retrieval mechanism for the position owner.

```shell
leftoverAdd = _maybeSecondPass(
    p.dex,
    pool,
    p.token0,
    p.token1,
    p.tickLower,
    p.tickUpper,
    p.tokenId,
    leftoverAdd,
    baseUSDC,
    remainingLossUSDCFinal

);
```

Recommendation

To ensure consistency, the team should account for any pending dust amounts and credit them to the positions whose liquidity is being increased. This ensures accurate tracking of value and prevents discrepancies in position balances.

UQA - Unconstrained Quoted Amount

Criticality	Minor / Informative
Location	UniswapAdapter.sol#L872,949 AerodromeAdapter.sol#L862,923
Status	Unresolved

Description

The swap execution functions set the `sqrtPriceLimitX96` parameter to zero when interacting with Uniswap V3 and Aerodrome routers. This parameter defines a price boundary that restricts how far the pool price can move during a swap. By setting it to zero, this safeguard is effectively disabled, allowing swaps to proceed without any constraint on price movement.

```shell
ISwapRouterV3.ExactInputSingleParams memory p =
ISwapRouterV3.ExactInputSingleParams({
    tokenIn: tokenIn,
    tokenOut: tokenOut,
    fee: fee,
    recipient: recipient,
    amountIn: amountIn,
    amountOutMinimum: minOut,
    sqrtPriceLimitX96: 0

});
```

Although the contract enforces slippage protection through the `amountOutMinimum` parameter, the lack of a defined price boundary exposes the swap to potentially significant price impact, as there is no limit on how far the pool price can shift during execution.

Recommendation

Consider whether implementing a price boundary based on the expected Time-Weighted Average Price (TWAP) could offer meaningful additional protection for large swaps, especially in pools with lower liquidity. This approach may help mitigate the impact of price manipulation or sudden volatility.

ZVE - Zero Value Estimation

Criticality	Minor / Informative
Location	Valuation.sol#L220,247,519,670
Status	Unresolved

Description

The valuation functions are designed to return zero in edge cases, such as when the input amount is extremely small or when quote calculations yield no output. This behavior avoids reverts but introduces a silent failure mode, where a non-zero valuation might be expected but is instead suppressed without warning.

```shell
function usdcValue(address dex, address token, uint256
amount) external view returns (uint256) {
    if (amount == 0) return 0;
    ...

    if (outScaled == 0) return 0;
    ...

}
```

Similarly, internal quote functions also return zero in such scenarios:

```shell
function _quoteThroughPool(...) internal view
returns (uint256) {
    if (amountIn == 0) return 0;
    ...

}
```

This approach can obscure the distinction between an actual zero valuation and a valuation process that failed silently. As a result, downstream logic that depends on these valuations may misinterpret the outcome, potentially leading to incorrect accounting or flawed interpretations.

Recommendation

The team is advised to consider whether critical valuation paths should revert when encountering unexpected zero outputs, rather than returning zero silently. Reverting in such cases allows callers to handle failures explicitly, reducing the risk of misinterpreting a zero return value as a valid result.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	Valuation.sol#L33,34,39,40,503 UniswapAdapter.sol#L22,23,24,25,26,34 RebalancePlanner.sol#L77,81,84,90,93,95 CLManager.sol#L42,45,48,53,57,58,59,60 CLCore.sol#L46,47 AerodromeAdapter.sol#L22,23,24,25,26,32
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```shell
address public USDC
address public CORE
uint32 public TWAP_SECONDS = 60
int24 public DEPTH_TICKS = 50


...
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.
Find more information on the Solidity documentation https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
Valuation	Implementation	Ownable, ReentrancyGuard		
		Public	✓	Ownable
	setCore	External	✓	onlyOwner
	setTWAPSeconds	External	✓	onlyOwner
	setDepthTicks	External	✓	onlyOwner
	refreshAll	External	✓	onlyOwner nonReentrant
	_uniqueFactoriesFromPools	Internal		
	usdcValue	External		-
	getBestRoute	External		-
	_bestRouteCached	Internal		
	_connectors	Internal		
	_isConnector	Internal		
	_refreshTokenToConnectors	Internal	✓	
	_bestEdgePoolAndScore	Internal	✓	
	_edgePoolScoreUSDC	Internal		
	_connectorUsdcPool	Internal		
	_edgePoolScoreUSDCExternal	External		-
	_quoteRoute	Internal		

	_poolCandidates	Internal		
	_uniswapPool	Internal		
	_aerodromePool	Internal		
	_poolAllowed	Internal		
	_poolTokens	Internal		
	_ctx	Internal		
	_aerodromeSpacings	Internal		
	_bridgeTokens	Internal		
	_quoteThroughPool	Internal		
	_twapSqrtPriceX96	Internal		
UniswapAdapter	Implementation	Ownable, ReentrancyGuard, Pausable, ICLDexAdapter		
		Public	✓	Ownable
	pause	External	✓	onlyGuardian
	unpause	External	✓	onlyGuardian
	setManager	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	setCore	External	✓	onlyOwner
	setTwapSeconds	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	getQuoter	External		-
	getNPM	External		-
	getFactory	External		-

	isAerodrome	External		-
	bestAllowlistedLiqScore	External		-
	validateAndGetPoolParams	External		-
	seedPairFromUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	mintPosition	External	✓	nonReentrant whenNotPaused onlyManager
	addLiquidity	External	✓	nonReentrant whenNotPaused onlyManager
	removeLiquidityBpsUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToTokens	External	✓	nonReentrant whenNotPaused onlyManager
	unwindToTokens	External	✓	nonReentrant whenNotPaused onlyManager
	swapExactUSDCForToken	External	✓	nonReentrant whenNotPaused onlyManager
	swapExactInToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	quoteToToken	External	✓	-
	getExpectedOutUSDC	External		-
	_getBestPath	Internal		
	_swapUSDCToToken	Private	✓	
	_swapTokenToUSDC	Private	✓	
	_swapExactInSingleHop	Private	✓	
	_executeRebalance	Private	✓	

	_quoteExactInSingleHop	Private	✓	
	_bestFee	Internal		
	_poolFor	Internal		
	_poolFee	Internal		
	_getTokensFromPosition	Internal		
	_poolTokens	Internal		
	_counterpartToken	Internal		
	_minOutFromPool	Internal		
	_minOutFromExpected	Internal		
	_expectedOutFromPool	Internal		
	_computeSlippageMins	Internal		
	_bestAllowlistedLiqScore	Internal		
	_requireBalance	Internal		
	_poolAllowed	Internal		
	_isBridgeToken	Internal		
	_usdcValue	Internal		
	_expectedOutUSDC	Internal		
	_lossBpsFromBudget	Internal		
	_minOutFromLossBudget	Internal		
	_usedLossUSDC	Internal		
	_clampSlippage	Internal		
	_approveExact	Internal	✓	
	_convertDustToUSDC	Internal	✓	
	_returnDust	Internal	✓	
	_collectFeesToTokensInternal	Internal	✓	

RebalancePlanner	Implementation	Ownable		
		Public	✓	Ownable
	setCore	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	setSolverIterations	External	✓	onlyOwner
	setEarlyStop	External	✓	onlyOwner
	planFromTokenBundle	External		-
	_solveIterative	Internal		
	_loadPoolContext	Internal		
	_safeGetSqrtPrice	Internal		
	_poolAllowed	Internal		
	_isBridgeToken	Internal		
ProtocolReserve	Implementation	Ownable, Pausable, ReentrancyGuard		
		Public	✓	Ownable
	pause	External	✓	onlyOwner
	unpause	External	✓	onlyOwner
	usdcBalance	External		-
	getShares	External		-
	getShareholders	External		-
	setShares	External	✓	onlyOwner whenNotPaused

	sweepReserves	External	✓	onlyOwner whenNotPaused nonReentrant
ICLDexAdapter	Interface			
	validateAndGetPoolParams	External		-
	seedPairFromUSDC	External	✓	-
	mintPosition	External	✓	-
	addLiquidity	External	✓	-
	removeLiquidityBpsUSDC	External	✓	-
	getExpectedOutUSDC	External		-
	unwindToTokens	External	✓	-
	swapExactUSDCForToken	External	✓	-
	swapExactInToUSDC	External	✓	-
	collectFeesToUSDC	External	✓	-
	collectFeesToTokens	External	✓	-
	quoteToToken	External	✓	-
	bestAllowlistedLiqScore	External		-
	getNPM	External		-
	getQuoter	External		-
	getFactory	External		-
	isAerodrome	External		-
ICLCore	Interface			
	isPoolAllowed	External		-
	isBridgeToken	External		-

	bridgeTokens	External		-
	listAllowedPools	External		-
	allowedDexes	External		-
	listAllowedDexes	External		-
	positionValueUSDC	External		-
	positionValueUSDCSingle	External		-
	pendingFees	External		-
	getPositionDetails	External		-
	getPosition	External		-
	USDC	External		-
	VALUATION	External		-
	protocolFeeBps	External		-
	botFeeBps	External		-
	protocolReserve	External		-
	positions	External		-
	allowedBots	External		-
	zeroFeeWallets	External		-
	isPoolDeprecated	External		-
	registerPosition	External	✓	-
	deregisterPosition	External	✓	-
	updateTokenMetadata	External	✓	-
	adjustTotalDeposited	External	✓	-
	setBotAllowedForPosition	External	✓	-
	addDustToPosition	External	✓	-
	withdrawDustForPosition	External	✓	-

	returnPosition	External	✓	-
	spotAmounts	External		-
ICLManager	Interface			
	protocolFeeBps	External		-
	botFeeBps	External		-
IValuation	Interface			
	usdcValue	External		-
	getBestRoute	External		-
IRebalancePlanner	Interface			
	planFromTokenBundle	External		-
INonfungiblePositionManager	Interface			
	positions	External		-
	mint	External	✓	-
	increaseLiquidity	External	✓	-
	decreaseLiquidity	External	✓	-
	collect	External	✓	-
	burn	External	✓	-
	ownerOf	External		-
	safeTransferFrom	External	✓	-
	transferFrom	External	✓	-

	setApprovalForAll	External	✓	-
ISlipstreamNonfungiblePositionManager	Interface			
	positions	External		-
	mint	External	✓	-
	increaseLiquidity	External	✓	-
	decreaseLiquidity	External	✓	-
	collect	External	✓	-
	burn	External	✓	-
	ownerOf	External		-
	safeTransferFrom	External	✓	-
	setApprovalForAll	External	✓	-
IUniswapV3Factory	Interface			
	getPool	External		-
IAerodromeFactory	Interface			
	getPool	External		-
	tickSpacings	External		-
	tickSpacingToFee	External		-
IUniswapV3Pool	Interface			
	liquidity	External		-

	token0	External		-
	token1	External		-
	fee	External		-
	tickSpacing	External		-
	factory	External		-
	slot0	External		-
	observe	External		-
	ticks	External		-
	feeGrowthGlobal0X128	External		-
	feeGrowthGlobal1X128	External		-
ICLPool	Interface	IUniswapV3 Pool		
ISlipstreamPool	Interface			
	slot0	External		-
ISwapRouterV3	Interface			
	exactInputSingle	External	Payable	-
ISlipstreamSwapRouter	Interface			
	exactInputSingle	External	Payable	-
IUniswapQuoterV2Like	Interface			
	quoteExactInputSingle	External	✓	-

ISlipstreamQuoterV2Like	Interface			
	quoteExactInputSingle	External	✓	-
ISlipstreamPoolState	Interface			
	feeGrowthGlobal0X128	External		-
	feeGrowthGlobal1X128	External		-
	ticks	External		-
	liquidity	External		-
CLManager	Implementation	ReentrancyGuard, Pausable, Ownable		
		Public	✓	Ownable
	setPlanner	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	setMinimumOpenUSDC	External	✓	onlyOwner
	setMaxBatchKeys	External	✓	onlyOwner
	setSecondPassParams	External	✓	onlyOwner
	pause	External	✓	onlyGuardian
	unpause	External	✓	onlyGuardian
	_requireOwnerOrBot	Internal		
	openPosition	External	✓	nonReentrant whenNotPaused

	exitPosition	External	✓	nonReentrant whenNotPaused
	returnNft	External	✓	nonReentrant
	collectFeesToUSDC	External	✓	nonReentrant whenNotPaused
	compoundFees	External	✓	nonReentrant whenNotPaused
	_compoundOne	Internal	✓	
	addCollateral	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	removeCollateral	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	changeRange	External	✓	nonReentrant onlyKeyOwner OrBot whenNotPaused
	allowBotForPosition	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	withdrawDust	External	✓	nonReentrant onlyKeyOwner whenNotPaused
	_addDust	Internal	✓	
	_quotedPositionValue	Internal		
	_corePositionValue	Internal		
	_tokenValueUSDC	Internal		
	_isZeroFee	Internal		
	_protocolFeeConfig	Internal		
	_protocolFeeAmount	Internal		

	_lossBudgetUSDC	Internal		
	_maybeSecondPass	Internal	✓	
	_handleBotFeeOnCollected	Internal	✓	
CLCore	Implementation	Ownable, IERC721Receiver, Pausable, ReentrancyGuard, ICLCore		
		Public	✓	Ownable
	setValuation	External	✓	onlyOwner
	setProtocolReserve	External	✓	onlyOwner
	setManager	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	addAllowedDex	External	✓	onlyOwner
	removeAllowedDex	External	✓	onlyOwner
	listAllowedDexes	External		-
	addBot	External	✓	onlyOwner
	removeBot	External	✓	onlyOwner
	setPoolStatus	Public	✓	onlyOwner
	setBridgeTokens	External	✓	onlyOwner
	setZeroFeeWallet	External	✓	onlyOwner
	setProtocolFeeBps	External	✓	onlyOwner
	setBotFeeBps	External	✓	onlyOwner
	pause	External	✓	onlyGuardian
	unpause	External	✓	onlyGuardian

	registerPosition	External	✓	nonReentrant onlyManager whenNotPaused
	deregisterPosition	External	✓	nonReentrant onlyManager whenNotPaused
	addDustToPosition	External	✓	nonReentrant onlyManager whenNotPaused
	withdrawDustForPosition	External	✓	nonReentrant onlyManager whenNotPaused
	returnPosition	External	✓	nonReentrant onlyManager
	updateTokenMetadata	External	✓	nonReentrant onlyManager whenNotPaused
	adjustTotalDeposited	External	✓	nonReentrant onlyManager whenNotPaused
	setBotAllowedForPosition	External	✓	nonReentrant onlyManager whenNotPaused
	isPoolAllowed	External		-
	listAllowedPools	External		-
	bridgeTokens	External		-
	isBridgeToken	Public		-
	listUserPositionKeys	External		-
	_nextKey	Internal	✓	
	getPosition	External		-
	getPositionDetails	Public		-
	listUserPositionDetails	External		-

	positionValueUSDC	External		-
	positionValueUSDCSingle	External		-
	spotAmounts	External		-
	pendingFees	External		-
	_pendingFeesForPosition	Internal		
	_positionValueUSDCSingle	Internal		
	_subUint256	Internal		
	_pendingFeesCalc	Internal		
	_registerPosition	Internal	✓	
	_deregisterPosition	Internal	✓	
	_positionPool	Internal		
	_spotAmountsForPosition	Internal		
	_liquidityOf	Internal		
	_pool	Internal		
	isPoolDeprecated	External		-
	_isPoolAllowed	Internal		
	_readSlot0	Internal		
	_valueTokenToUSDC	Internal		
	onERC721Received	External		-
AerodromeAdapter	Implementation	Ownable, ReentrancyGuard, Pausable, ICLDexAdapter		
		Public	✓	Ownable
	pause	External	✓	onlyGuardian

	unpause	External	✓	onlyGuardian
	setManager	External	✓	onlyOwner
	setGuardian	External	✓	onlyOwner
	setCore	External	✓	onlyOwner
	setTwapSeconds	External	✓	onlyOwner
	setValuation	External	✓	onlyOwner
	getQuoter	External		-
	getNPM	External		-
	getFactory	External		-
	isAerodrome	External		-
	bestAllowlistedLiqScore	External		-
	validateAndGetPoolParams	External		-
	seedPairFromUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	mintPosition	External	✓	nonReentrant whenNotPaused onlyManager
	addLiquidity	External	✓	nonReentrant whenNotPaused onlyManager
	removeLiquidityBpsUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	collectFeesToTokens	External	✓	nonReentrant whenNotPaused onlyManager
	unwindToTokens	External	✓	nonReentrant whenNotPaused onlyManager

	swapExactUSDCForToken	External	✓	nonReentrant whenNotPaused onlyManager
	swapExactInToUSDC	External	✓	nonReentrant whenNotPaused onlyManager
	quoteToToken	External	✓	-
	getExpectedOutUSDC	External		-
	_getBestPath	Internal		
	_swapUSDCToToken	Private	✓	
	_swapTokenToUSDC	Private	✓	
	_swapExactInSingleHop	Private	✓	
	_executeRebalance	Private	✓	
	_quoteExactInSingleHop	Private	✓	
	_bestTickSpacing	Internal		
	_poolFor	Internal		
	_poolTickSpacing	Internal		
	_getTokensFromPosition	Internal		
	_poolTokens	Internal		
	_counterpartToken	Internal		
	_minOutFromPool	Internal		
	_minOutFromExpected	Internal		
	_expectedOutFromPool	Internal		
	_computeSlippageMins	Internal		
	_bestAllowlistedLiqScore	Internal		
	_requireBalance	Internal		
	_poolAllowed	Internal		

	_isBridgeToken	Internal		
	_usdcValue	Internal		
	_expectedOutUSDC	Internal		
	_lossBpsFromBudget	Internal		
	_minOutFromLossBudget	Internal		
	_usedLossUSDC	Internal		
	_clampSlippage	Internal		
	_approveExact	Internal	✓	
	_convertDustToUSDC	Internal	✓	
	_returnDust	Internal	✓	
	_collectFeesToTokensInternal	Internal	✓	

Inheritance Graph

The inheritance graph for the EZManager contracts can be accessed through the link.

Flow Graph

The flow graph for the EZManager contracts can be accessed through the link.

Summary

The EZManager protocol proposes a liquidity management system, providing modular tracking, accounting, and valuation of LP positions across Uniswap V3 and Aerodrome Slipstream. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io